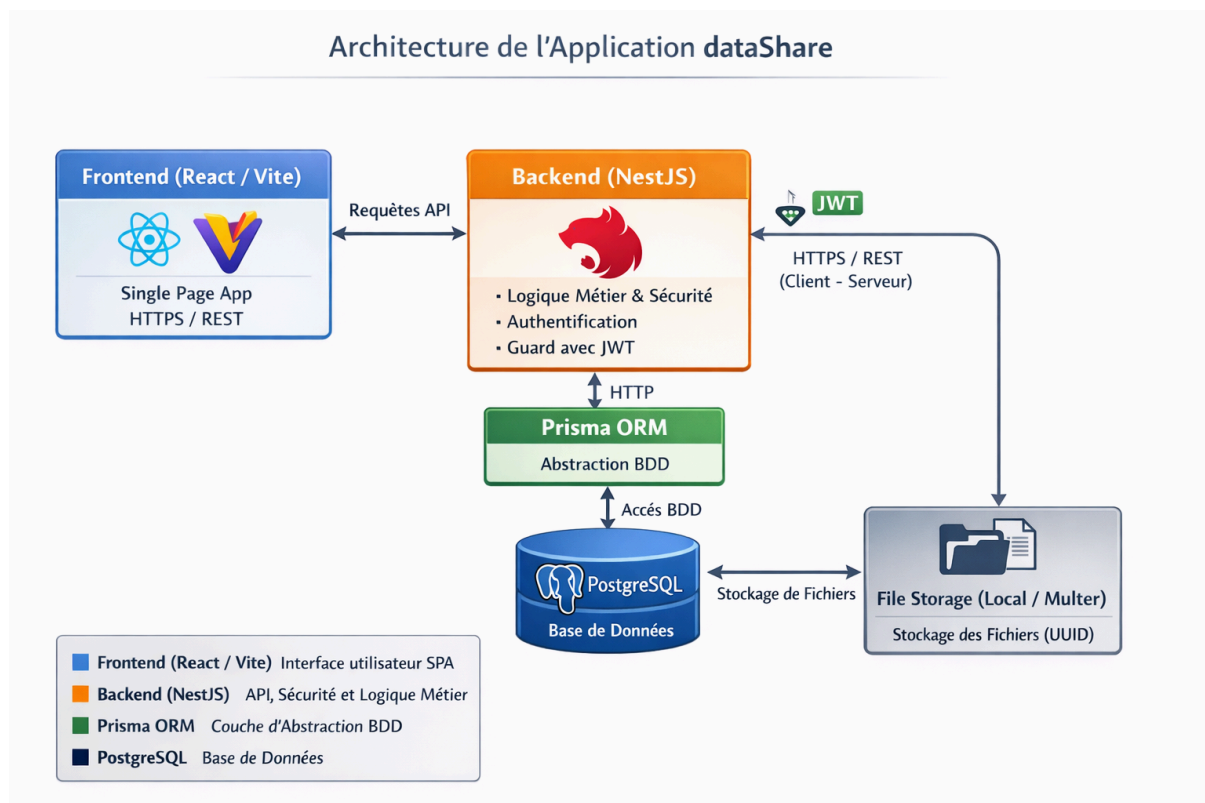


Documentation Technique - dataShare

1. Architecture de l'application

L'application **dataShare** repose sur une architecture découplée (Client/Serveur) facilitant la maintenance et le déploiement.



Légende :

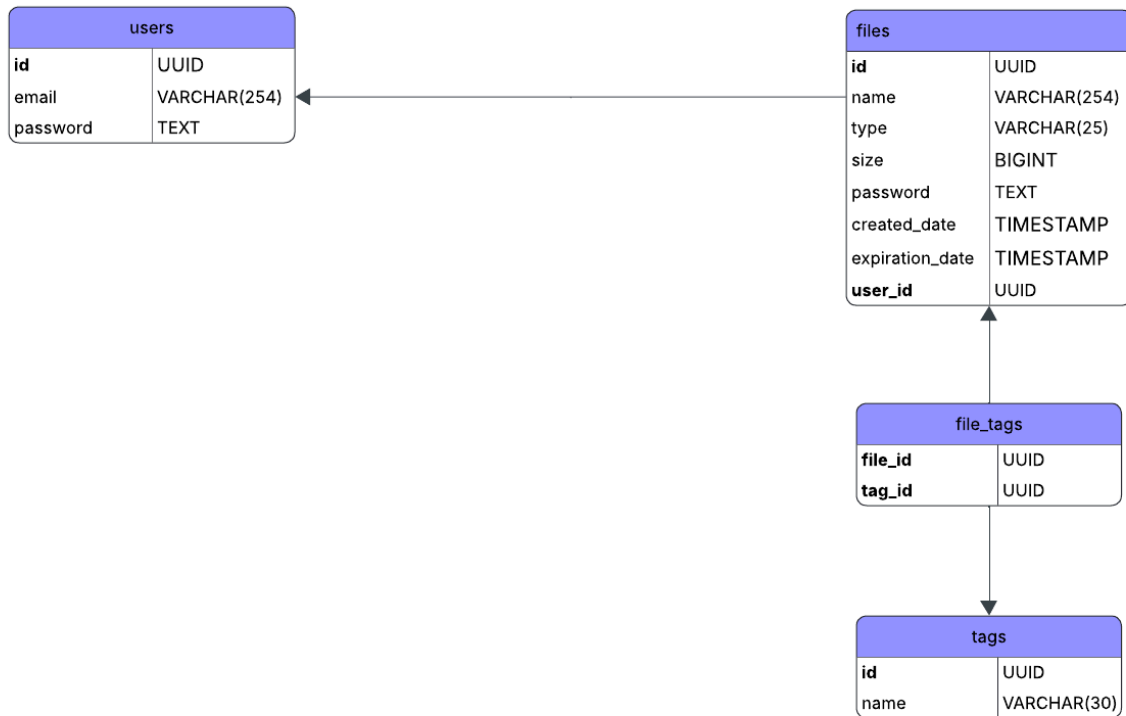
- **Frontend (React/Vite)** : Interface utilisateur SPA communiquant via HTTPS/REST.
 - **Backend (NestJS)** : API gérant la logique métier, la sécurité et l'orchestration des données.
 - **Prisma ORM** : Couche d'abstraction pour les échanges avec la base de données.
 - **PostgreSQL** : Persistance des métadonnées (utilisateurs, fichiers, dates d'expiration).
 - **File Storage (Local/Multer)** : Stockage physique des fichiers sur le serveur (renommés avec des UUID).
-

2. Choix technologiques justifiés

<i>Élément</i>	<i>Technologie choisie</i>	<i>Alternatives</i>	<i>Justification</i>
<i>Langage</i>	<i>TypeScript</i>	<i>JavaScript, Java</i>	<i>Typage statique pour réduire les erreurs en production et meilleure autocomplétion.</i>
<i>Frontend</i>	<i>React (Vite)</i>	<i>Angular, Vue</i>	<i>Écosystème riche, rapidité de développement et performance via le Virtual DOM.</i>
<i>Backend</i>	<i>NestJS</i>	<i>Express Fastify</i>	<i>Architecture modulaire robuste, support natif de TypeScript et de l'injection de dépendances.</i>
<i>Base de données</i>	<i>PostgreSQL</i>	<i>MongoDB</i>	<i>Données relationnelles (User <-> Files) nécessitant une intégrité transactionnelle forte.</i>
<i>ORM</i>	<i>Prisma</i>	<i>TypeORM, Sequelize</i>	<i>Typage automatique basé sur le schéma et facilité de migration.</i>
<i>Authentification</i>	<i>Passport-JWT</i>	<i>Sessions, OAuth2</i>	<i>Approche "Stateless" idéale pour une API, facilitant le passage à l'échelle.</i>

3. Modèle de données

Le schéma de données est centré sur la relation entre les utilisateurs et leurs fichiers.



Règles de gestion principales :

- Un utilisateur possède plusieurs fichiers.
- Un fichier appartient à un seul utilisateur (**user_id**).
- Le champ **password** du fichier est optionnel (protection de lien).
- Le status du fichier (valide/expiré) est calculé pour faciliter le filtrage.

4. Documentation d'API

L'API est documentée via **Swagger (OpenAPI)**.

- **Accès local** : <http://localhost:3000/api/docs>
- **Endpoints principaux** :
 - [POST /auth/register](#) & [POST /auth/login](#) : Gestion des comptes.
 - [GET /files](#) : Liste des fichiers de l'utilisateur (filtrable).
 - [POST /files/upload](#) : Téléversement avec Multer (limite 1 Go).
 - [DELETE /files/:id](#) : Suppression sécurisée.

5. Sécurité et gestion des accès

La sécurité est intégrée à chaque couche de l'application :

- **Authentification JWT** : Utilisation d'un **AuthGuard** global. Le client doit fournir un header `Authorization: Bearer <token>`.
 - **Autorisations (RBAC/Ownership)** : Le **FileOwnerGuard** vérifie systématiquement que l'ID de l'utilisateur connecté correspond au champ `user_id` de la ressource demandée.
 - **Sécurisation des fichiers** :
 - Renommage via `UUID` pour empêcher la devinette d'URLs.
 - Hashage des mots de passe de fichiers et des utilisateurs.
 - Limite stricte de 1 Go via `Multer`.
 - **Mesures serveurs** : **Helmet** pour les headers HTTP et **CORS** restreint à l'origine du frontend.
 - **Nettoyage** : Un `Cron Job` automatique supprime les fichiers dont l'expiration date est dépassée.
-

6. Qualité, tests et maintenance

Le projet suit une pyramide de tests pour assurer la stabilité :

- **Tests Unitaires & Intégration (Backend)** : Couverture de 75% via **Jest** et **Supertest** (Logique métier et cycle de vie des fichiers).
 - **Tests UI (Frontend)** : Validation des composants avec **React Testing Library**.
 - **Tests E2E (Cypress)** : Simulation d'un parcours utilisateur complet (Inscription -> Login -> Upload -> Download -> Delete -> Logout).
 - **Maintenance** : Nettoyage automatique des données expirées toutes les heures. Utilisation de **class-validator** pour assurer l'intégrité des données entrantes.
-

7. Processus d'installation et d'exécution

Prérequis : Docker & Docker Compose.

1. **Configuration** : Créer un fichier `.env` dans le dossier `/server` (`JWT_SECRET`, `DATABASE_URL`).
 2. **Installation et Lancement** :
 - `docker-compose up --build`
 3. **Commandes de tests** :
 - Client : `npm run test` (Jest) / `npm run test:e2e` (Cypress).
 - Serveur : `npm run test` / `npm run test:integration`.
-

8. Utilisation de l'IA dans le développement

L'intelligence artificielle a été intégrée comme un partenaire de "pair programming" tout au long du cycle de vie du projet :

- **Copilotage pour l'implémentation** : L'IA a été sollicitée pour structurer la logique complexe d'upload côté serveur (intégration de Multer dans NestJS, gestion des flux de fichiers). Chaque itération générée a fait l'objet d'une **review humaine rigoureuse** pour s'assurer du respect des bonnes pratiques NestJS et de la sécurité des types.
- **Aide au debug** : Lors des erreurs critiques (notamment sur les surcharges de types TypeScript), l'IA a servi d'outil de diagnostic rapide, permettant d'identifier des incohérences de code que le compilateur ne signalait pas explicitement.
- **Supervision humaine** : Le rôle de l'IA est resté consultatif : toute logique métier sensible (comme les Guards de propriété de fichiers) a été revue et débuggée manuellement pour garantir qu'aucune faille de sécurité n'était introduite par l'automatisation.